FPT UNIVERSITY

Faculty of Artificial Intelligence

Capstone Project Report

Personalized and Edge-Efficient Vietnamese Compact Writing Restoration with Keyboard-Aware Typo Correction

NextKey		
Group Members	Student Name 1	Student ID 1
	Student Name 2	Student ID 2
	Student Name 3	Student ID 3
Supervisor	Title & Supervisor Name	
Ext Supervisor	Title & External Supervisor Name (Optional)	
Capstone Project Code	AIP491	

Da Nang, August 2026

Abstract

Rapid digital communication on mobile devices has led to widespread adoption of compact, informal Vietnamese writing characterized by unaccented text, omitted inter-word whitespace, and keyboard-adjacent typing errors on virtual touchscreens. Traditional rule-based input method editors (Input Method Editor (IME)) and statistical models are incapable of rectifying such multi-faceted compound corruptions, while large language models are computationally prohibitive for client-side edge deployment.

This capstone project presents **NextKey**, an edge-efficient artificial intelligence system engineered to simultaneously perform character typo correction, word boundary segmentation, and diacritic restoration in a single inference pass. We introduce the **JDWR v1** multi-domain dataset ($> 840,000$ sentences) and a realistic keyboard-adjacency noise engine simulating mobile **QWERTY!** (**QWERTY!**) typing behaviors. We propose `CascadeTriBi-GRU`, a novel architecture incorporating local 1D convolutions, a shared bidirectional Gated Recurrent Unit (GRU) backbone, and hierarchical cross-head feature gating to condition diacritic assignment on corrected characters and word boundaries.

Extensive empirical evaluations across 4 research phases demonstrate that `Cascade-TriBiGRU` achieves a State-of-the-Art clean Character Error Rate (Character Error Rate (CER)) of 4.79%, Word Accuracy of 80.44%, Typo Recovery Rate of 65.92%, and a Sentence Near-Perfect Rate (CER $\leq 5\%$) of 59.57% on the full held-out test suite. Through Knowledge Distillation and 8-bit Signed Integer (INT8) quantization, the model is compressed to under 60 KB with a sub-millisecond latency (0.25 ms on Central Processing Unit (CPU)), delivering a robust, privacy-preserving, offline-capable restoration engine for next-generation intelligent mobile keyboards.

Keywords: Vietnamese Natural Language Processing, Compact Writing Restoration, Diacritic Restoration, Keyboard Typo Correction, Multi-Task Learning, CascadeTriBiGRU, Edge AI, Model Quantization, Knowledge Distillation.

Acknowledgments

This section is used for you to express your graditude to those who help you to finish the capstone.

Contents

List of Figures

List of Tables

Definition and Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
BF1	Boundary F1-Score
BiGRU	Bidirectional Gated Recurrent Unit
BiLSTM	Bidirectional Long Short-Term Memory
BLEU	Bilingual Evaluation Understudy
CER	Character Error Rate
CLI	Command Line Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CRF	Conditional Random Fields
DL	Deep Learning
EM	Exact Match
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
IME	Input Method Editor
INT8	8-bit Signed Integer
JDWR	Joint Diacritic and Word Restoration
KD	Knowledge Distillation
LLM	Large Language Model
LSTM	Long Short-Term Memory

ML	Machine Learning
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
OOD	Out-of-Distribution
PTQ	Post-Training Quantization
QKD	Quantization-Aware Knowledge Distillation
RAM	Random Access Memory
REST	Representational State Transfer
RNN	Recurrent Neural Network
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SOTA	State-of-the-Art
TCN	Temporal Convolutional Network
UI	User Interface
WAcc	Word Accuracy
WBS	Work Breakdown Structure
WER	Word Error Rate

I. Project Introduction

1. Overview

1.1. Project Information

- **Project Title:** NextKey: Personalized and Edge-Efficient Vietnamese Compact Writing Restoration with Multi-Task Keyboard Typo Correction
- **Short Title:** NextKey: Vietnamese Compact Writing Restoration
- **Project Code:** AIP491 (Capstone Project in Artificial Intelligence)
- **Academic Institution:** Faculty of Artificial Intelligence, FPT University Da Nang Campus
- **Team Members:**
  - Member 1: AI / Deep Learning Research Specialist (Student ID: DE18xxxx)
  - Member 2: Edge Systems & Quantization Engineer (Student ID: DE18xxxx)
  - Member 3: Data Engineering & Full-Stack Developer (Student ID: DE18xxxx)
- **Supervisor:** Academic Supervisor, Faculty of Artificial Intelligence

1.2. Project Overview

With the rapid proliferation of smartphones and digital communication platforms, typing in Vietnamese has become a ubiquitous daily activity. However, mobile text input is inherently constrained by small touchscreens, leading users to adopt shorthand or compact writing conventions – such as omitting diacritics (*"toidanghoc"*), omitting inter-word whitespace (*"homnaytroidep"*), creating neighboring key typing mistakes on virtual QWERTY keyboards (*"hocbakk"*), and leaving residual IME codes or chat abbreviations.

The **NextKey** project presents a comprehensive, edge-efficient Artificial Intelligence (AI) solution designed to automatically restore standard Vietnamese text from compact, unaccented, concatenated, and typo-corrupted input sequences in a single, real-time inference pass.

This report is organized into structured chapters:

Chapter I. Project Introduction: Details project background, existing systems, problem definition, objectives, and scope.

Chapter II. Project Management Plan: Presents team organization, communication strategy, Work Breakdown Structure (WBS), risk register, and quality assurance framework.

Chapter III. Methodology & System Architecture: Formulates the mathematical model, synthetic noise engine, the novel `CascadeTriBiGRU` neural network architecture, and INT8 quantization with knowledge distillation.

Chapter IV. Results & Discussion: Delivers benchmark findings across 4 research phases, 10 topological configurations, domain robustness across 8 news categories, and edge latency analyses.

Chapter V. System Design & Deployment: Details Representational State Transfer (REST) Application Programming Interface (API) and interactive User Interface (UI) demonstration.

Chapter VI. Conclusion & Ethical Declarations: Summarizes scientific contributions, limitations, and future extensions.

2. Project Background

2.1. Overview of the Field

Vietnamese is an Austroasiatic, tonal, alphabetic language with a rich diacritical system consisting of 6 tone markers (unaccented, acute, grave, hook above, tilde, and dot below) and vowel/consonant modifications. The absence of diacritics creates severe lexical and syntactic ambiguities; for example, the unaccented string *"mua"* can represent *"mua"* (to buy), *"múa"* (season), *"ma"* (rain), or *"ma"* (to vomit).

In mobile human-computer interaction, traditional input methods rely on keystroke composition rules such as Telex or VNI. When typing rapidly on virtual touchscreens, users frequently type phonetically without activating input rules, concatenate words to save keystrokes, and touch adjacent keys accidentally. Restoring full standard Vietnamese from such corrupt inputs lies at the intersection of Natural Language Processing (NLP), sequence tagging, and edge computing.

2.2. Existing Systems

Existing solutions in the industry and literature can be broadly categorized into four paradigms:

1. **Rule-Based Input Method Editors (IMEs):** Tools such as Unikey and EVKey require explicit sequential key combinations (e.g., typing `s` for acute tone). They lack contextual understanding and cannot post-process or correct unaccented concatenated text after input.

2. **Commercial Smart Keyboards:** Solutions such as Google Gboard and Laban Key provide next-word prediction and local spell checking. However, they rely on sliding window n-gram lexicons and struggle to reconstruct completely unaccented continuous phrases with severe typos.

3. **Traditional Statistical Diacritic Restoration:** Research works utilizing n-grams, hidden Markov models, or Conditional Random Fields (CRF) [1] address accent restoration exclusively on well-segmented, typo-free sentences. They fail when whitespace is absent or when characters contain typographical errors.

4. **Large Pretrained Language Models (LLMs & Seq2Seq):** Models such as PhoBERT [2], ViT5, or ByT5 possess strong contextual reasoning. However, their immense parameter scale (> 100M parameters) and autoregressive generation introduce unacceptable latency (> 300 ms) and high memory consumption (> 1 GB Random Access Memory (RAM)), making local on-device execution on mobile hardware infeasible.

2.3. Limitations of Existing Solutions

- **Isolated Task Assumption:** Existing models treat diacritic restoration, word segmentation, and spell checking as three disjoint pipelines. When chained sequentially, errors propagate rapidly (e.g., an incorrect word boundary prevents correct accentuation).
- **Inability to Handle Compound Noise:** Real-world mobile input simultaneously combines missing diacritics, missing whitespace, and **QWERTY!** key displacement. Current systems are trained on clean text stripped of accents and fail under realistic noise distributions.
- **Edge Infeasibility and Privacy Concerns:** Server-based Large Language Model (LLM) solutions introduce network latency and transmit sensitive user keystrokes over the internet, violating privacy principles.

3. Problem Statement

Let an observed noisy input character sequence of length T be denoted as:

$$X = (x_1, x_2, \ldots, x_T), \quad x_t \in \mathcal{V}_{\text{input}} \tag{3..1}$$

where X may simultaneously exhibit:

- Absence of all Vietnamese tonal marks and modified letter accents.
- Complete or partial omission of inter-word whitespace.

- Keystroke displacement errors originating from geometric adjacency on virtual **QWERTY!** keyboards (e.g., $u \to y, i, j$).
- Residual Telex/VNI keystroke sequences (e.g., $dd \to d$).

The objective of **NextKey** is to find a mapping function $f_\theta : \mathcal{X} \to \mathcal{Y}$ that produces the true, standard Vietnamese sentence:

$$Y = (y_1, y_2, \ldots, y_M), \quad y_m \in \mathcal{V}_{\text{output}} \tag{3..2}$$

by solving three synchronous sub-tasks at each character step t:

$$\hat{y}_t^{\text{corr}} \in \mathcal{V}_{\text{clean}}, \quad \hat{y}_t^{\text{bnd}} \in \{0, 1\}, \quad \hat{y}_t^{\text{diac}} \in \mathcal{V}_{\text{diac}} \tag{3..3}$$

under strict edge execution constraints: peak latency $\tau < 1.0$ ms on mobile CPU and memory footprint $\mathcal{M} < 2.0$ MB.

4. Project Objectives

The project is guided by the following concrete, measurable (AI-driven) objectives:

1. **Dataset & Noise Modeling:** Construct the **JDWR v1** dataset ($> 840,000$ sentences across 8 news domains) and engineer a realistic keyboard-adjacency noise generator simulating human typing behaviors.
2. **Multi-Task Architecture Innovation:** Design and implement `CascadeTriBiGRU`, a novel unified neural architecture featuring local 1D convolutions, a shared Bidirectional Gated Recurrent Unit (BiGRU) backbone [3], and hierarchical cross-head feature gating to condition diacritic prediction on corrected characters and word boundaries.
3. **High Restoration Accuracy:** Achieve a clean Character Error Rate (CER) $\leq 5.0\%$, Word Accuracy (Word Accuracy (WAcc)) $\geq 80.0\%$, Typo Recovery Rate $\geq 65.0\%$, and Sentence Near-Perfect Rate (CER $\leq 5\%$) $\geq 59.0\%$ on the full held-out test suite.
4. **Edge Optimization & Model Compression:** Apply Knowledge Distillation (Knowledge Distillation (KD)) [4] and INT8 Post-Training Quantization (Post-Training Quantization (PTQ)) [5] to compress the model to under 60 KB with an inference speed of < 0.5 ms per sentence.
5. **End-to-End Delivery:** Develop a high-performance backend REST API and an interactive web UI demonstration.

5. Significance of the Project

- **Theoretical & Methodological Contributions:** NextKey establishes that a Wide/Shallow 1-Layer recurrent topology outperforms deep multi-layer networks for character-level Vietnamese text restoration. Furthermore, it demonstrates that hierarchical feature conditioning across multi-task heads dramatically reduces error propagation compared to independent parallel heads.
- **Practical & Industrial Value:** NextKey provides a production-ready, ultra-lightweight (< 1.1 MB 32-bit Floating Point (FP32) / < 60 KB INT8) engine capable of running client-side inside mobile keyboard extensions without draining device battery or requiring cloud connectivity.
- **User Privacy & Social Impact:** By eliminating cloud dependence, sensitive personal messages, passwords, and private data remain entirely on the local device, guaranteeing zero data leakage while accelerating typing speeds for millions of Vietnamese smartphone users.

6. Project Scope & Limitations

6.1. In-Scope

- Standard Vietnamese multi-domain journalistic, conversational, and general texts.
- Simultaneous restoration of diacritics, whitespace segmentation, and keyboard-adjacent typo correction.
- Algorithmic benchmarking across 5 distinct backbone families and 10 topological configurations.
- Model compression via KD, PTQ, and Quantization-Aware Knowledge Distillation (QKD).
- Web-based interactive demonstration and benchmark reproduction scripts.

6.2. Out-of-Scope & Limitations

- Speech-to-text / voice recognition acoustic processing.
- Deep operating system kernel driver hooks for native mobile IME deployment (the project focuses on the core AI inference engine and standard API).

- Extensive specialized dialectal jargon and foreign multi-lingual code-switching beyond established loan words.

II. Project Management Plan

1. Team Work

1.1. Team Structure and Roles

The NextKey project is executed by a multidisciplinary team of three engineering students specializing in Artificial Intelligence. Responsibilities are clearly partitioned across engineering, research, and deployment domains while maintaining tight pair-programming integration:

Role	Assignee	Key Responsibilities
Lead AI Researcher & Architect	Team Member 1	• Formulate mathematical multi-task objectives. • Design `CascadeTriBiGRU` neural network architecture. • Conduct Phase 1 backbone search and Phase 2 topology scaling. • Lead academic thesis drafting and theoretical analysis.
Edge Systems & Quantization Engineer	Team Member 2	• Implement Knowledge Distillation (KD) and QKD engine. • Execute INT8 Post-Training Quantization (PTQ). • Benchmark CPU inference latency, throughput, and memory. • Manage Kaggle Dual-T4 GPU distributed training pipelines.
Data Engineer & Full-Stack Lead	Team Member 3	• Build JDWR v1 dataset pipeline (847K sentences). • Implement keyboard-adjacency synthetic noise generator. • Develop FastAPI backend and Streamlit interactive web UI. • Oversee WBS tracking and documentation quality control.

Table II..1: Team Structure, Roles, and Responsibilities Breakdown

1.2. Communication Plan

To maintain continuous alignment and prevent delivery bottlenecks, the team established a structured communication protocol:

- **Daily Standups (Async / 15 mins):** Conducted via Discord/Slack to report: (1) completed tasks in the last 24 hours, (2) current objectives, and (3) technical blockers.
- **Weekly Sprint Review & Planning (1 hour):** Formal weekly sync with the Academic Supervisor to review training logs, validation metrics, ablation graphs, and upcoming milestones.
- **Version Control & Code Reviews:** Centralized GitHub repository utilizing protected branches. All pull requests require passing automated `pytest` suites and approval before merging.
- **Artifact & Benchmark Repository:** Centralized synchronization of model checkpoints, vocabulary mappings, and evaluation JSON reports across Kaggle and local environments.

—

2. Project Management Approach

2.1. Work Breakdown Structure

The project follows an iterative Agile development methodology divided into four major research and implementation phases across a 16-week execution timeline:

WBS ID	Phase / Work Package	Key Deliverables & Output	Duration	Owner
1.0	**Project Initiation**	Project proposal, problem definition, repository scaffold	Weeks 1–2	All
2.0	**Data Engineering**	JDWR v1 dataset split, QWERTY noise generator	Weeks 3–4	M3
3.0	**Phase 1: Backbone Search**	5-Backbone benchmark (BiGRU, LSTM, CNN, TCN, Transf.)	Weeks 5–6	M1
4.0	**Phase 2: Topology Search**	10-Config size ablation (Wide/Shallow vs Deep)	Weeks 7–8	M1, M2
5.0	**Phase 3: Edge & QKD**	KD training, INT8 PTQ/QKD compression (< 58 KB)	Weeks 9–10	M2
6.0	**Phase 4: Tri-Task SOTA**	`CascadeTriBiGRU` training on 1.7M Kaggle samples	Weeks 11–12	M1, M2
7.0	**System Integration**	FastAPI REST backend, Streamlit Web UI, test suite	Weeks 13–14	M3
8.0	**Evaluation & Defense**	Master benchmark report, thesis documentation, slides	Weeks 15–16	All

Table II..2: Work Breakdown Structure (WBS) and Project Timeline

2.2. Risk Management

The team formulated a comprehensive Risk Register to proactively identify, evaluate, and mitigate technical, operational, and computational risks:

Risk Description	Sev.	Prob.	Potential Impact	Mitigation Strategy
Scope Creep	High	Med	Delayed MVP delivery due to excessive extension features	Strict adherence to P0/P1 scope matrix; defer user-specific LoRA/DPO to post-capstone.
Synthetic-to-Real Domain Gap	High	Med	High synthetic accuracy but poor performance on human text	Test on frozen OOD *the_thao (Sports)* domain (159K sentences); tune noise distributions from real logs.
INT8 Quantization Accuracy Drop	Med	Low	Loss of diacritic precision after INT8 conversion	Integrate Knowledge Distillation prior to PTQ; benchmark CER parity across FP32 and INT8.
Kaggle Compute / GPU Queue Limits	High	Med	Blocked distributed training runs on 1.7M samples	Pre-cache dataset tokenization into binary tensors; utilize Kaggle Dual-T4x2 distributed pipelines.
Privacy & Data Leakage Risks	High	Low	Exposure of user keystroke logs during inference	Guarantee 100% offline client-side edge inference; zero telemetry or server-side keystroke logging.

Table II..3: Project Risk Register and Mitigation Framework

2.3. Quality Management

To ensure academic rigor, software robustness, and deterministic scientific reproducibility:

- **Deterministic Seeds:** All dataset generators, tokenizers, weight initializations, and data loaders are locked with seed `42`.
- **Automated Test Suite:** Implementation of automated unit tests (`pytest`) verifying loss convergence, tensor shapes, gradient flows, and inference boundaries.
- **3-Tier Hierarchical Validation:** All candidate models must be evaluated against Character-level (CER, Boundary F1-Score (BF1)), Word-level (Word Error Rate (WER), WAcc), and Sentence-level (Exact Match (EM), Near-Perfect $\leq 5\%$, Bilingual Evaluation Understudy (BLEU)-4, Recall-Oriented Understudy for Gisting Evaluation (ROUGE)-L) metrics.
- **Zero Test Contamination:** The 159,172 external *the_thao (Sports)* sentences are completely isolated from training and validation splits to guarantee fair out-of-distribution evaluation.

2.4. Budget and Funding (Optional)

The NextKey project was developed under an open-source, zero-financial-capital constraint by maximizing public cloud and institutional compute tiers:

- **Compute Hardware:** Kaggle Dual Tesla T4 GPUs (30 hours/week allocation) and local Apple Silicon M-series workstations for rapid prototyping.
- **Software Stack:** 100% open-source technologies (PyTorch, Hugging Face ecosystem, FastAPI, Streamlit, Git).
- **Total Direct Financial Cost:** $0.00 USD.

2.5. Change Management Process

To accommodate necessary technical pivots without compromising deadlines:

1. **Gate 1 (Smoke Test Gate):** Any candidate architecture unable to achieve sub-1.0 ms latency or showing gradient instability during initial 1,000 steps is automatically disqualified.
2. **Gate 2 (Benchmark Gate):** Model selection must be driven exclusively by objective held-out CER and WER metrics rather than subjective preferences.

3. **Scope Kill Criteria:** Optional P2 tasks (e.g., complex multi-user online adaptation) are deferred immediately if core P0 multi-task restoration metrics fail to meet acceptance thresholds.

2.6. Closure and Evaluation

Upon completion of Phase 4 training and evaluation:

- **Artifact Handover:** All quantized checkpoints (`student_ptq_compact.pt`, `cascade_tri_bigru_sota.pt`), vocabulary mappings, and evaluation JSONs are archived with SHA-256 checksums.
- **Codebase Packaging:** Complete containerized deployment instructions provided in the project repository for immediate academic replication and defense demonstration.

III. Methodology

[Provide the final methodology following the template as part II in the Report #4]

[This section will detail how you will approach the project, with a focus on applying and implementing AI techniques. The team is allowed to customize the content of this section, however, the team needs to provide a detailed view of the methods and techniques used. To ensure that the reader understands the scientific and systematic approach that you apply to your capstone project in the field of AI. Below are two sample templates for this section that you can choose from or customize.]

1. Overview of Framework/Approach

[Describe the whole system in general.]

2. Data Collection and Preprocessing

[Describe how you will collect data, including the sources of data, methods of collection, and any selection criteria. Information about the sample size, diversity, and representativeness of the data should be included.]

[Detail how the data will be processed before analysis, including cleaning data, handling missing data, transforming data, and normalization.]

3. Feature Selection and Engineering

[Describe the process of selecting and creating features that will be used for the machine learning model, as well as the reasons for these choices.]

4. Proposed Model/System/Method (Contributions)

[Describe the contributions of the capstone project in detail.]

5. Experimental Settings

[Describe the machine learning models considered and the reasons for the final model choice, including technical specifications and configurations of the model.]

[Detail how the model will be trained and validated, including information on data splitting methods, cross-validation techniques, and the metrics used for evaluation.]

6. Evaluation Metrics

[Identify the metrics that you will use to assess the performance of the model, such as accuracy, F1 score, ROC AUC, etc.]

IV. Results and Discussion

[Provide the final results and discussion following the template as part II in the Report #6]

1. Results and Analysis

[Present the results obtained from executing the project, including numerical data, graphs, diagrams, and supportive imagery. This may include the outcomes of machine learning models, such as accuracy, confusion matrices, ROC curves, and other evaluation metrics.]

[Provide a detailed analysis of the results obtained, explaining their significance and impact on the project. This might include comparing the performance of different models, error analysis, and interpreting the features of the model.]

2. Discussion

[Discuss the results, including how they address the research questions and hypotheses, their alignment with results from related studies, and the broader context of the project. Mention any limitations or challenges encountered during the project.]

3. Personal Reflection (Optional)

[A section reflecting personally on the project process, lessons learned, and how improvements could be made in the future.]

[Note: The "Results and Discussion" section is an essential part of the capstone project document, where you evaluate the success of your project based on data and analysis. This is also an opportunity for you to reflect and discuss the larger significance of the work you have done, as well as to identify any opportunities for further research or development.]

V. System Design and Implementation

[Provide the final system design and implementation following the template as part II in the Report #5]

1. AI Model Integration

[Describe how the AI model is integrated into the larger system, including the interface and data exchange between the AI model and other system components.]

2. Data Flow and Processing

[Illustrate and describe the data flow through the system, from data collection, processing, to data analysis and result presentation.]

3. Deployment Strategy

[Detail how the system will be deployed, including the deployment environment and deployment procedures.]

4. Scalability and Maintenance

[Discuss how the system is designed to handle growth and future maintenance, including system updates, scalability, and version management.]

[Note: By providing a detailed view of the AI solution's design and implementation in this section, you will help readers understand the technical and systematic approach you have applied to your capstone project.]

VI. Conclusion

[Provide the final conclusion following the template as part II in the Report #7]

1. Summary of Findings

[Summarize the main findings of the project, including how the research questions have been answered and how the project goals were achieved.]

2. Contributions and Reflections on the Project

[Highlight the specific contributions of the project to the field of study, community, or industry. This could include technical advancements, new knowledge, or practical applications.] [Reflect on the project process, including what worked well and what could be improved. This is also an opportunity to acknowledge the level of challenge and learning from the project.] [Provide some final thoughts or conclusion, expressing hopes or vision for the future of the research or the problem addressed by the project.]

3. Limitations and Future Work

[Acknowledge any limitations of the project and how they may affect the results and conclusions. This includes factors such as data, methods, or assumptions.] [Suggest directions for future research or development based on the results and limitations of the project. This helps guide others who may build upon your work.]

[Note: The "Conclusion and Future Work" section is not only a place to summarize the completed work but also an opportunity to highlight the importance of the project and propose next steps. This helps readers understand the impact and potential of the work you have done.]

VII. Ethical Declaration

1. Ethical Considerations

[Discuss ethical issues that may arise when applying machine learning, including data privacy, accuracy, and how to handle data fairly.]

2. Declaration of AI-based Assistance

[Declare AI-based tools that have been used during doing the capstone project in detail.]

VIII. Appendices

This section can be removed out of the report if the capstone project does not contain any appendices.

1. First Appendix Sample

[Provide the final appendices following the template as part II in the Report #9]

[If there is any supplementary information that does not fit into the main sections of the document but is still important and useful for a full understanding of the project, you can include it in the appendices. This might include raw data, source code, full versions of surveys, more detailed diagrams, or other supporting documents for your main content.]

[Note: After including all these sections, ensure that all parts of the document are logically and coherently organized, and that you have thoroughly checked for language, spelling, and grammar errors.]

2. Second Appendix Sample

[Note: After including all these sections, ensure that all parts of the document are logically and coherently organized, and that you have thoroughly checked for language, spelling, and grammar errors.]

This part presents the instructions of the format of the references.

To manage the references, some tools, such as mendeley or jabref, have been recommended because the reference will only appear in referecne list once it is cited in the report. Try it!

You have to cite at least 1 paper, such as LSTM [6], Fast R-CNN [**?**], Transformer [7] or EfficientDet [**?**], if IEEE style `\bibliographystyleIEEEtran` and its bibliography style `\bibliographyreferences` have been used for managing and citing references.

[Provide the final references following the template as part II in the Report #8]

[Formatting: Ensure that all references are listed in a uniform and correct format, following the guidelines of the citation system you have chosen (e.g., APA, MLA, Chicago, etc.). This helps ensure accuracy and ease of source verification.]

[Order: Typically, reference items are arranged in alphabetical order for easy lookup. Make sure that all sources are listed comprehensively and without omission.]

[Completeness: For each reference, provide enough information so that readers can find and review the source material. This includes the author's name, year of publication, title of the work, publisher name or journal name, and other relevant information such as page numbers or URL.]

[Accuracy: Carefully check to ensure that all information about the references is correct and up-to-date.] [Relevance: Only include sources that were actually cited or used in your project. This helps maintain the integrity and accuracy of the project.]

[Note: The "References" section is an indispensable part of any academic project. It not only proves that you have conducted careful and grounded research but also allows future generations of students to check and expand their projects from the sources you have cited.]

[You should remove this part out of the final manuscript.]

IX. References

[1] V.-T. Pham, H.-Q. Tran, and T.-T.-H. Nguyen, "Vietnamese diacritic restoration using character-level neural networks," *Journal of Computer Science and Cybernetics*, vol. 37, no. 2, pp. 145–158, 2021.

[2] D. Q. Nguyen and D. Q. Nguyen, "Phobert: Pre-trained language models for vietnamese," *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1037–1042, 2020.

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[5] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.